

REVIEW

Open Access



A systematic analysis on performance and computational complexity of sorting algorithms

Sureshkumar Sundaramoorthy^{1†} and Govardhan Karunanidhi^{1*†}

[†]Sureshkumar Sundaramoorthy and Govardhan Karunanidhi have contributed equally.

*Correspondence: Govardhan Karunanidhi
kgovardhan@vit.ac.in

¹School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, Tamilnadu, India

Abstract

With the exponential growth of digital data, efficient sorting techniques has becomes essential for real-time and large-scale computing applications. This review systematically evaluates the performance and computational complexity of twelve classical sorting techniques. This study considers both theoretical aspects, such as time, space complexity, and algorithmic stability, and complements them with empirical analysis using the MATLAB platform. Execution times were benchmarked across a variety of data types, including positive and negative integers, decimal numbers, and characters, over data sizes ranging from 100 to 100,000 elements. Results indicate that the bucket sort delivers the fastest performance for uniformly distributed numeric and character data, while the counting sort excels with positive integers. Merge and heap sort provide stable and scalable performance across data volumes due to their consistent $O(n \log n)$ complexity. In contrast, simpler sorting techniques, such as bubble and selection sorting, show significant inefficiency on larger data sizes. This study bridges the gap between theoretical analysis and empirical behavior, offering practical insight to help researchers and developers select optimal sorting techniques based on specific application needs and input characteristics.

Keywords Sorting algorithms, Flowcharts, Data structure, Performance, Space complexity, Time complexity

1 Introduction

Sorting is the one of the most fundamental and ubiquitous operations in the field of computer technology. It is the technique of ordering a group of elements in the correct sequence, like numbers/alphabets. There are various sorting criteria that can be used, the most popular being the sorting of numbers and/or strings either in ascending or descending order. Based on the necessity, a suitable and necessary criteria can be selected by the user [1, 2]. Several applications, including databases, search engines, and machine learning, rely on sorting algorithms. To solve sorting difficulties, a wide variety of sorting techniques exist. The majority of software applications require data



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

sorting. When sorting, we have multiple input elements, like $i_1, i_2, i_3 \dots i_n$. The output elements must be defined as $i_1 \leq i_2 \leq i_3 \dots \leq i_n$, or $i_1 \geq i_2 \geq i_3 \dots \geq i_n$, to arrange them in ascending or descending order. However, this isn't always necessary, data elements may be sorted in either a non-linear or linear data structure, based on the type of information they represent [3–5]. Therefore, research in sorting algorithms is crucial to computer science. A variety of basic and enhanced sorting algorithm are available, that are efficient for specific sorting problems only. They depend on a number of factors, such as volume, kind, and format of the input data as well as potentially the specifications of the computer [6–9]. Further, there may be a lot of factors to be considered when picking a sorting technique to address a specific sorting issue [10–13]. In today's world, handling a huge amount of data on a computer is the main challenge, and a crucial step in that process is sorting. Sorting improves the efficiency of searching for particular data among the huge data volumes. Today, we have several sorting techniques. Especially when it comes to real-time data processing, sorting is a significant and extensively researched problem for which the computation's execution time and resources are crucial. Therefore, it's critical to evaluate and contrast its results with those of every other classification algorithm that is available [14–16]. This paper provides several types of sorting algorithms, from inefficient but comprehensible alternatives to programming language-based efficient algorithms. There are several sorting algorithms, and not all of them executes in the same manner. This paper tries to review and comprehensively report on their complexity to determine which sorting performance is optimal.

A notable gap exists in the comprehensive comparative analysis of well-established sorting techniques, which involves assessing their real-world performance across various data types and input sizes. Developers and researchers frequently encounter uncertainty when opting the most efficient sorting technique for particular uses, especially in situations where time is limited or resources are scarce. Existing literature primarily focuses on the theoretical complexity of sorting techniques or evaluates them under restrictive conditions. This study aims to close the gap through empirical analysis and benchmarking of various sorting methods within a uniform setting (MATLAB), encompassing diverse data types and scales. The outcomes are intended to assist practitioners in opting the most suitable sorting techniques based on real-world effectiveness metrics rather than solely relying on asymptotic evaluation [104].

MATLAB, generates random or structured datasets that facilitate the testing of algorithms. This ensures a comprehensive evaluation of their performance across different input scenarios. It employs timing functions to measure the execution time and compare their time complexities. This enables a comprehensive analysis of the strengths and weaknesses of each algorithm in various scenarios [103].

1.1 Related work and background

Numerous studies have examined the performance and computational complexity of sorting techniques, evaluating them across various platforms, data types, and implementation strategies. These comparative studies offer valuable insights but often focus on specific aspects, limiting their generalizability across data types and execution environments.

Roşca and Cărbureanu et al. [105] analyzed sorting algorithms implemented in different programming languages, highlighting how language-specific optimizations affect

runtime. Their work emphasized large-scale integer data but did not extend to diverse data types such as characters or floating-point numbers. Ala'Anzy et al. [106] presented a broad literature review of sorting algorithms from both theoretical and implementation perspectives. However, their study lacked empirical validation and did not account for data distribution variability. Rizvi et al. [107] focused on use-case relevance but were limited to standard datasets, omitting structural and type-based variation. Sabah et al. [108] evaluated performance based on dataset size and structure, offering a more practical perspective. However, their study centered on execution time alone, without discussing stability or space complexity, factors that are crucial for algorithm selection in constrained environments.

Several works have explored improvements to classical algorithms. Zutshi and Goswami et al. [109] introduced a hybrid enhancement to traditional quadratic-time sorts, where subarrays are identified and merged dynamically. Their technique outperformed basic bubble, insertion, and selection sort variants in targeted benchmarks. Similarly, Chauhan and Duggal et al. [110] empirically compared common algorithms and observed that insertion and selection sorts are well-suited to small or nearly sorted inputs, while merge and quick sort perform better on larger, randomized data. Gill et al. [111] extended this by emphasizing the scalability of merge, quick, and heap sort, supporting their suitability for large datasets due to their consistent $O(n \log n)$ performance. Buradagunta et al. [112] evaluated sorting algorithms using randomly generated integers under varied input conditions, showing that quick sort excelled with large volumes, while their custom UNH sort was efficient on smaller datasets.

Parallel and GPU-based sorting has also been an active area of research. Singh et al. [113] provided a foundational survey of GPU-accelerated sorting models, highlighting how parallelism can drastically improve throughput on large datasets. Alhabboub et al. [114] introduced a hybrid parallel sorting framework for multi-core systems, achieving significant speed-up through concurrent merge operations. Gupta et al. [115] proposed a GPU-based approach leveraging sorted matrix structures for improved memory and thread utilization. Kobus et al. [116] refined GPU sorting further by optimizing segmented sorting to boost warp-level execution efficiency. Meanwhile, Jalilvand et al. [117] surveyed hardware-based implementations, discussing trade-offs in sorting using FPGA, ASIC, and GPU systems.

While these studies offer valuable contributions, they often focus on specialized hardware, parallelism models, or language-specific behaviors. In contrast, our work provides a unified, empirically grounded comparison of 12 classical sorting algorithms under a consistent CPU-based environment using MATLAB. We evaluate performance across five dataset sizes and multiple data types (integers, decimals, characters), with an emphasis on execution time, complexity, stability, and space usage.

2 Sorting algorithms

The different sorting algorithms used widely are as follows:

2.1 Bubble sort

It is the simplest sorting technique that sorts the elements of an array by continuously shifting the biggest element to the maximum group index. Every element is compared to its neighboring element, and swapping is made when the first element value is bigger

than the value of next. This procedure is repeated starting from the initial element of an array to the whole array, unless all elements are sorted [17–19]. This technique halts when every element is in correct order with their neighboring elements and ensures consistency with synchronizing the whole array if no elements are swapped at the end of the iteration [20–22]. The below Fig. 1 shows steps for bubble sort.

2.1.1 Execution

It works by traversing the array from left to right. In each iteration, the indexed element is compared with the element next to it. Swapping of the elements takes place if the element in the current position is bigger than the neighbouring element. This iteration continues over the array and compare the elements unless a complete pass is achieved without the need for swaps. This procedure is continued until the whole array gets sorted [23–28]. Figure 2 flowchart describes the operation of bubble sort algorithm.

2.2 Bucket sort

A distributive-based sorting technique called bucket sort splits an array's elements into several buckets. After that, each bucket is sorted separately, either by recursively using the bucket sorting method or by employing any other sorting method. The final sorted array is obtained by concatenating the buckets. it is also referred as bin sort [29, 30]. This sorting technique is non-comparative, which implies it can sort the elements without comparing pairs of them. This can make bucket sort more efficient than comparison sorting algorithms for certain types of data, that is already partially sorted or data that has a limited range of values [31]. The below figure 3 shows steps for bucket sort.

2.2.1 Execution

Bucket sort initially identifies the lowest and highest values in the given unsorted array to determine its range. The array is divided based on its range values into a suitable number of buckets and create empty buckets based on the length of the array. The elements of a main array are distributed into several buckets based on their values, sorting every single bucket utilizing a distinct sorting technique, and finally each sorted bucket is concatenated to obtain a final array [32–35]. Figure 4 flowchart describes the operation of bucket sort algorithm.

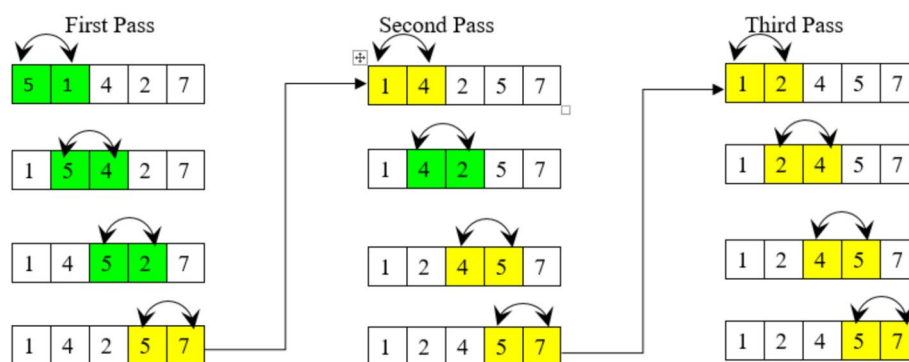


Fig. 1 Example Illustrating Bubble Sort Operation

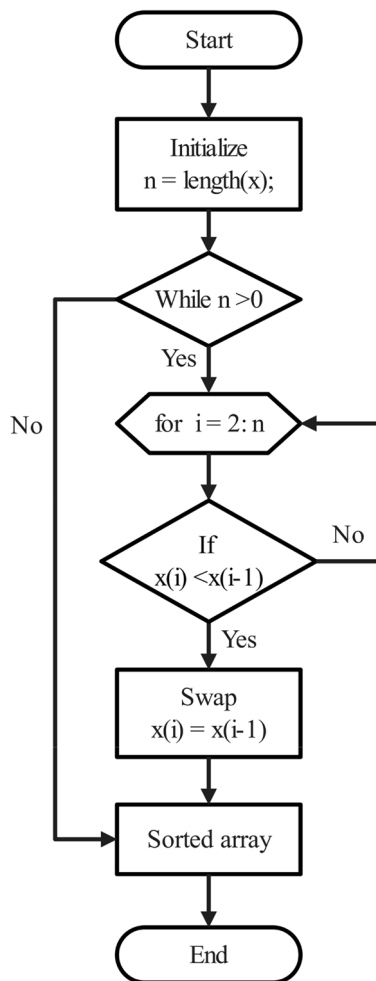


Fig. 2 Flowchart 1. Bubble Sort Algorithm

2.3 Cocktail sort

A variant of the bubble sorting technique which sorts an array bidirectionally is called cocktail sort, sometimes called bidirectional bubble sort or Shaker sort. It improves upon standard bubble sort by allowing elements to move both forward and backward through the array [36].

The first forward pass from left to right side of an array similar to bubble sort will shift the largest element to end of an array, and the subsequent first backward iteration will move the lowest element value to beginning of an array. The second-biggest and second-lowest element values shall be moved to their appropriate places in the array during the second complete iteration, and this procedure is repeated until the array has been sorted [37]. The below figure 5 shows steps for cocktail sort.

2.3.1 Execution

For execution, each iteration of this technique is broken down into two phases:

1. Firstly, loop traverse from left to right over the array, identical to bubble sort. During the loop, adjacent elements are compared and if the value on the left is greater than the value on the right, then values are swapped and the largest element will reside at the end of the array.

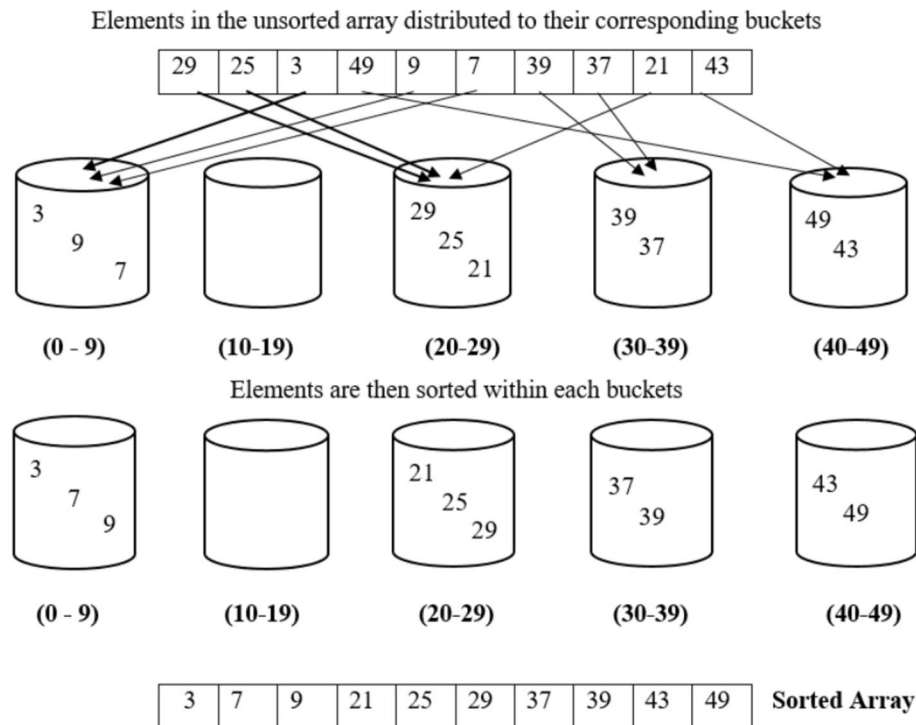


Fig. 3 Example Illustrating Bucket Sort Operation

2. Secondly, the loop traverses in the reverse direction, beginning with the recently sorted element at the end of the array. Adjacent elements are compared throughout the loop; element values are swapped when the right element value is smaller than the left element value, and the smallest element will reside at the beginning of the array [1, 2]. Figure 6 flowchart describes the operation of cocktail sort algorithm.

2.4 Comb sort

It's a comparative-based sorting technique that outperforms bubble sorting. It compares the array elements, which are far apart and gradually reducing the gap between each other to efficiently move smaller elements towards the beginning of the list. The fundamental concept is to apply a shrink factor to reducing gap between array elements being compared. Starting with largest gap value, the gap reduces by a factor of 1.3 for each iteration until it reaches one, where it behaves like a regular bubble sort. The unique part about comb sort is that it uses a larger initial gap, which helps to quickly remove turtles (smallest one close to the end of the array)[2].The below figure 7 shows steps for comb sort.

2.4.1 Execution

The algorithm starts with an initial gap value that is typically the length of the list being sorted. For each iteration of the sorting process, the gap is shrunk by a factor of 1.3. This means that the gap is effectively halved with each iteration. For each iteration, compare the elements, which are separated by the present gap. When elements are not in the correct sequence, swap them. Continue shrinking the gap and performing comparisons and swaps until the gap reaches 1. At this point, the array will be sorted[37]. Figure 8 flowchart describes the operation of comb sorting technique.

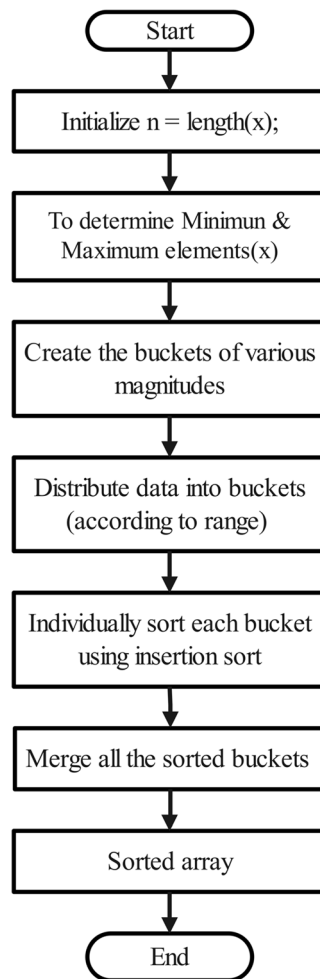


Fig. 4 Flowchart 2. Bucket Sort Algorithm

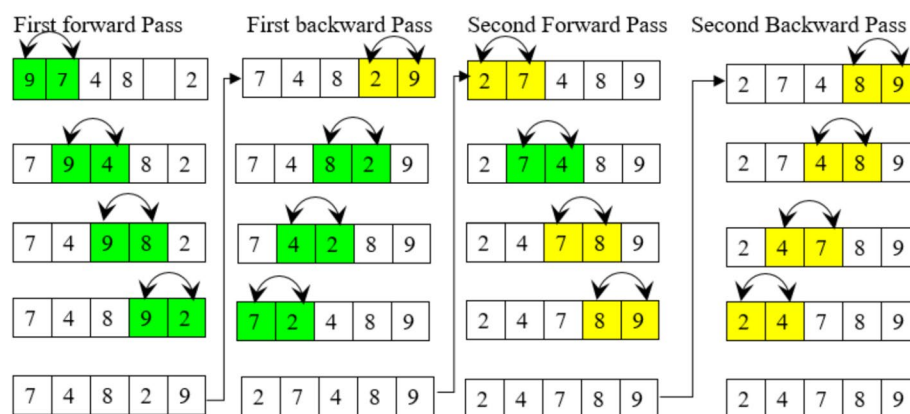


Fig. 5 Example Illustrating Cocktail Sort Operation

2.5 Counting sort

The counting sort technique is non comparative based, which sorts the array elements by counting the number of instances of every distinct element in the given data elements, creating a cumulative array from the count array, then performing sort operation

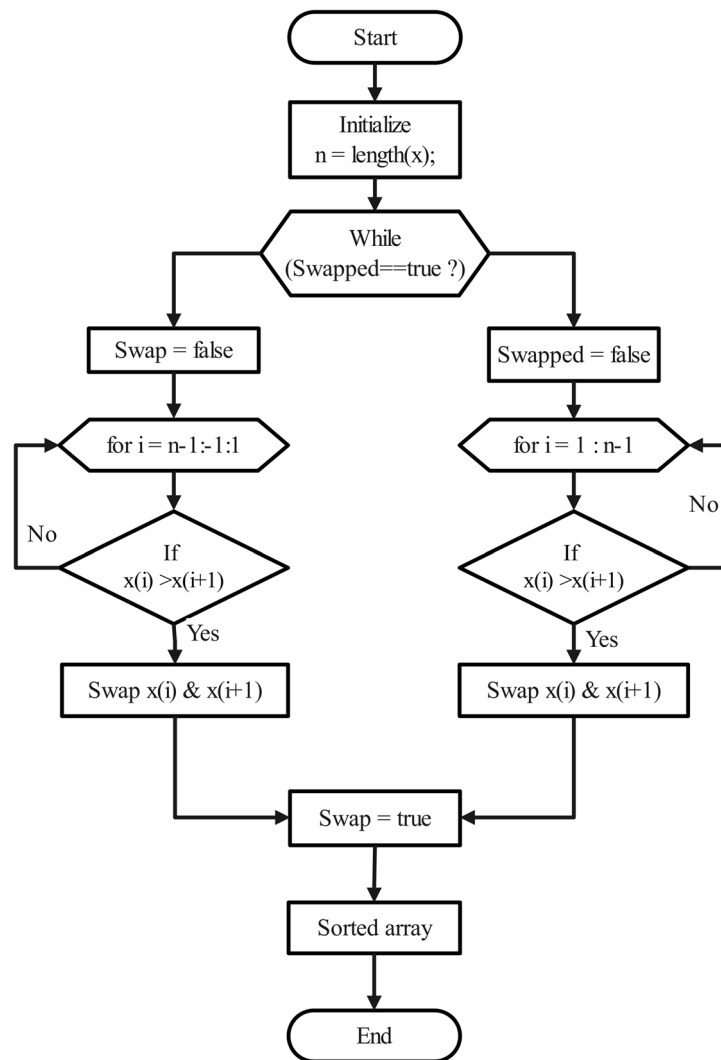


Fig. 6 Flowchart 3.Cocktail Sort Algorithm

for every element and placing it in its appropriate sorted position using that information [37, 38]. The below figure 9 shows steps for counting sort.

2.5.1 Execution

This technique executes by counting the number of instances of every unique element in the given data and storing these counts in a separate count array. Then creating a cumulative count array by adding each element count with the previous element count in the count array, after that creates an empty output array, which keeps sorted elements [26]. The given array is traversed again by using the count array and cumulative array to find the appropriate place for every element in the resultant array. The elements are placed in their determined positions and decreases the elements count in the count array in order to handle duplicates. After processing all elements, the output array contains the sorted elements [39–41]. Figure 10 flowchart describes the operation of counting sort algorithm.

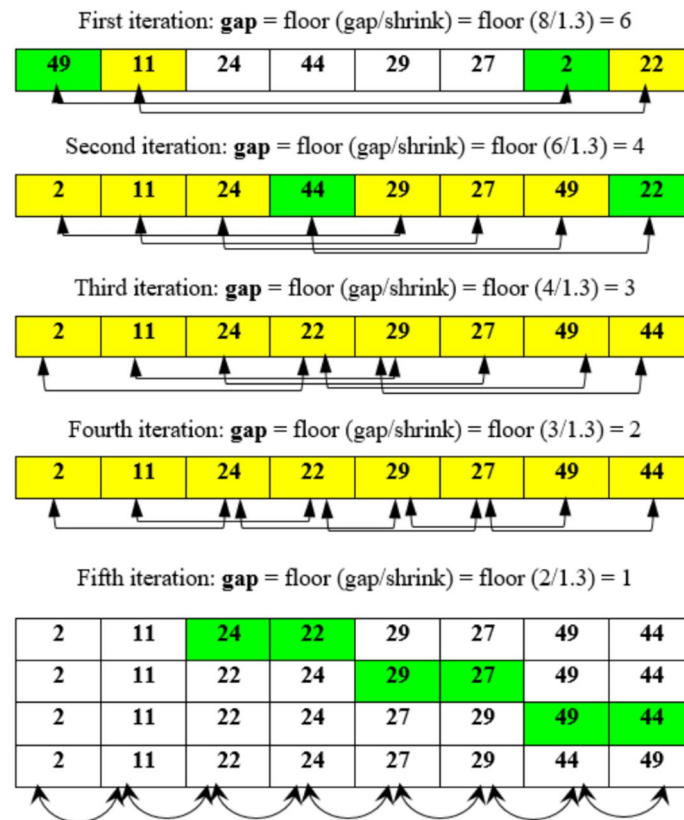


Fig. 7 Example Illustrating Comb Sort Operation

2.6 Heap sort

Heap sorting is an in-place, comparative, and unstable sorting technique that uses a binary heap-based data structure to sort an array. Initially, an array of elements is rearranged in the binary max-heap, which then continuously replaces the heap root with the final heap element, declaring that heap element as a part of a sorted suffix. Next, the heap was affected by the root replacement, which is fixed such that the biggest element is once again at its root. The above procedure continues until heap contains single element value [42]. It has two major steps: constructing the heap using data input and continuously removing the largest element from a heap and putting it in the sorted arrays last position [43]. The below figure 11 shows steps for heap sort.

2.6.1 Execution

The execution of this algorithm starts with converting the given array into a max-heap that ensures the biggest element is always found at the root and then starting from the bottom of a tree and moving upwards. Heapification is performed for each element in the array to maintain the max-heap property by swapping elements during heapification. The largest element (heap root) is removed and arranged at the end of arrays sorted section. After removing largest element, heapification is performed on the remaining elements to restore the max-heap property. Until the whole array is sorted, the above procedure is repeated [43–46]. Figure 12 flowchart describes the working of heap sorting technique.

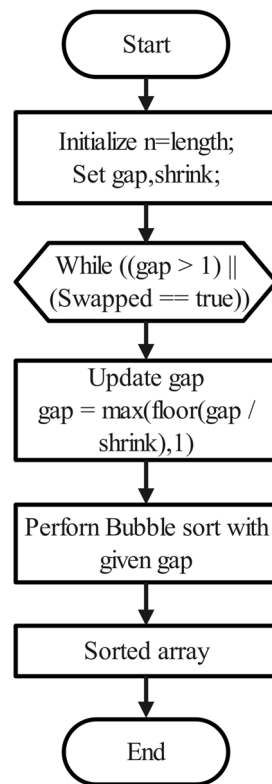


Fig. 8 Flowchart 4.Comb sort technique

2.7 Merge sort

A divide-and-conquer sort technique called merge sorting divides an array into two sub arrays, recursively sorts every sub array, and then combines the sorted sub arrays back to generate the resultant sorted array. This technique is recursive in nature; it divides an initial array into half-equal parts unless it can no longer be divided, implying that only one element remains [47–51]. The below figure 13 shows steps for merge sort.

2.7.1 Execution:

To implement merge sort algorithm below steps to be follows:

1. Splitting: Divide the unsorted array into two equal halves (i.e. sub arrays) until the sub arrays contains single element.
2. Sorting: Recursively sort the sub arrays created in the previous step. This recursion continues until all sub arrays contain only one element, as a single element is always considered as sorted.
3. Merging: Merge the sorted sub arrays to produce new sorted sub arrays. This process combines two sorted sub arrays into a single sorted array. The merging continues until a single sorted array is obtained, which represents the final sorted array [52–58]. Figure 14 flowchart describes the operation of merge sort algorithm.

2.8 Insertion sort

A simple sorting technique called "insertion sort" generates a sorted array iteratively using an unsorted array. Every single element from the unsorted array is compared with each element in the sorted array to determine its proper placement before being

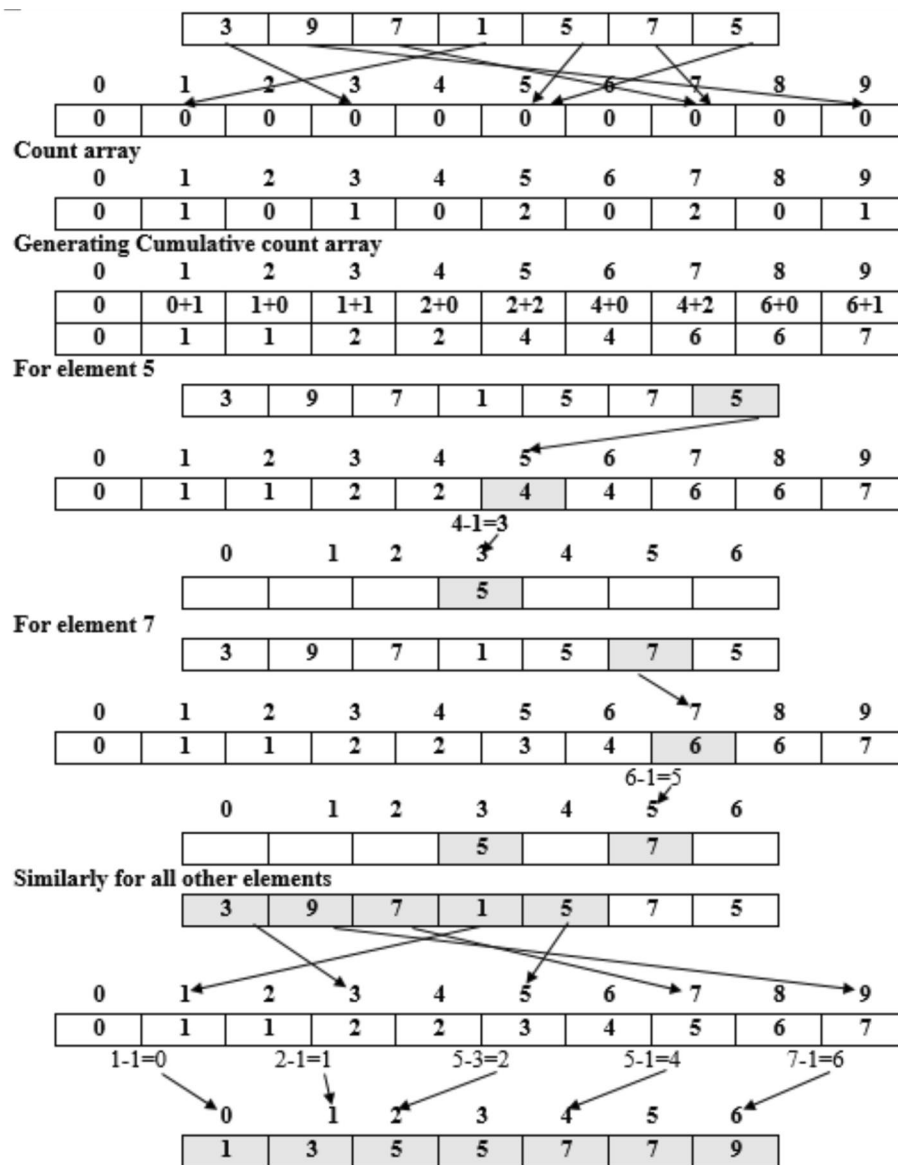


Fig. 9 Example Illustrating Counting Sort Operation

inserted. The above procedure continues if the whole array is to be sorted [18, 24, 25]. The initial assumption made by this technique is that the first element has already been sorted. Then, iteratively traversing through the unsorted section, it identifies the right spot for every element by contrasting them and moving them to the right [36, 59, 60]. The below figure 15 shows steps for insertion sort.

2.8.1 Execution

The algorithm starts with creating an empty array to store the sorted elements when progress across every element in the unsorted array, beginning with the second element because the first element is considered sorted initially. Select the current element from the unsorted array, treating it as the "key" to be inserted into the sorted array and then start comparing the key to the elements in the sorted array, starting from the rightmost element. If the sorted arrays current element is bigger than the key, shift it towards the

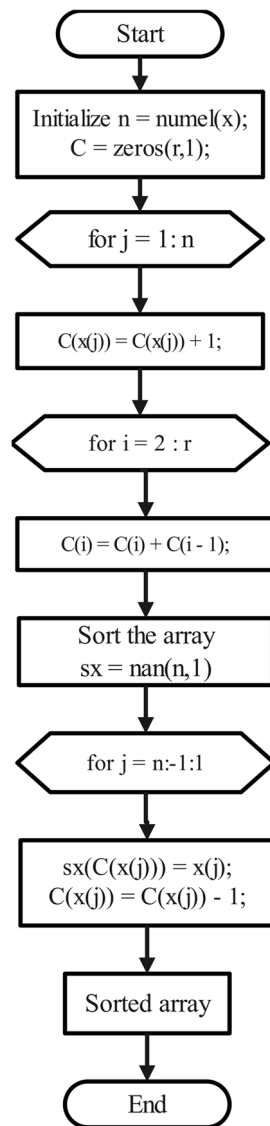


Fig. 10 Flowchart 5.Counting sort algorithm

right, creating a vacancy for the key and insert it into the sorted array at that location. Continue this process of selecting, comparing, shifting, and inserting elements until all unsorted elements have been placed in their correct position within the sorted array [43] [61–63]. Figure 16 flowchart describes the operation of insertion sort algorithm.

2.9 Quick sort

Quicksort is an extremely effective divide and conquering sorting approach that operates well on huge arrays. It operates by picking a pivot among the array elements and splitting the other array elements into two halves, based on whether their value is greater or lesser than a pivot. After that, the sub-arrays are recursively sorted [44, 47, 64]. The below figure 17 shows steps for quick sort.

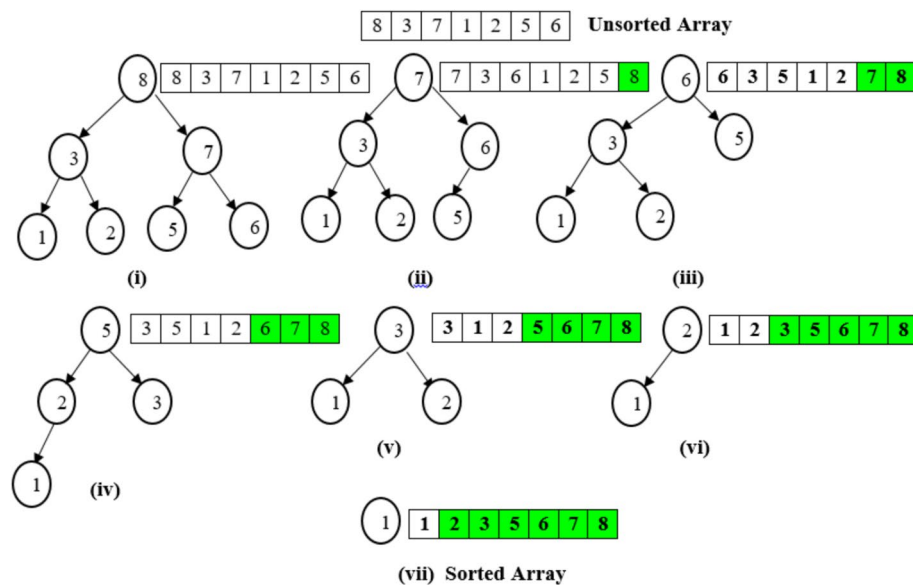


Fig. 11 Example Illustrating Heap Sort Operation

2.9.1 Execution

Picking a pivot element from the initial given array is the primary step of execution. The whole array will be segmented using this pivot element. Partitioning a given unsorted input array is an essential step in the quicksort technique. Placing a pivot in the right place within the sorted array is the goal of partitioning, and then placing the smaller elements on the pivot left and the larger elements on the pivot right. For picking pivots, there are several options.

- Select pivot to be either the last or first element.
- Choose a pivot element at random.
- Select a middle element as a pivot.

Once the pivot is positioned correctly, partitioning takes place recursively on both sides of the pivot, and then it sorts the whole array finally[64–66]. Figure 18 flowchart describes operation of quick sorting technique.

2.10 Radix sort

Radix is a linear, non-comparison, stable sorting technique that sorts array by traversing through every single-digit position, instead of simply comparing the elements. The elements of an array are dispensed into buckets depending on the value of each digit (radix). The resultant sorted array is obtained by continuously sorting the elements from least to most significant by their significant digits [2, 67]. The below figure 19 shows steps for radix sort.

2.10.1 Execution

This algorithm starts by identifying the highest element value in the array, and this value is used to calculate the number of runs needed to sort the data array and then determine how many digits are in the highest element. This will determine how many iterations are needed to sort the array, as each pass sort's one digit place. An array of buckets are created, where the number of buckets is equal to the base of the numbers being sorted (e.g.,

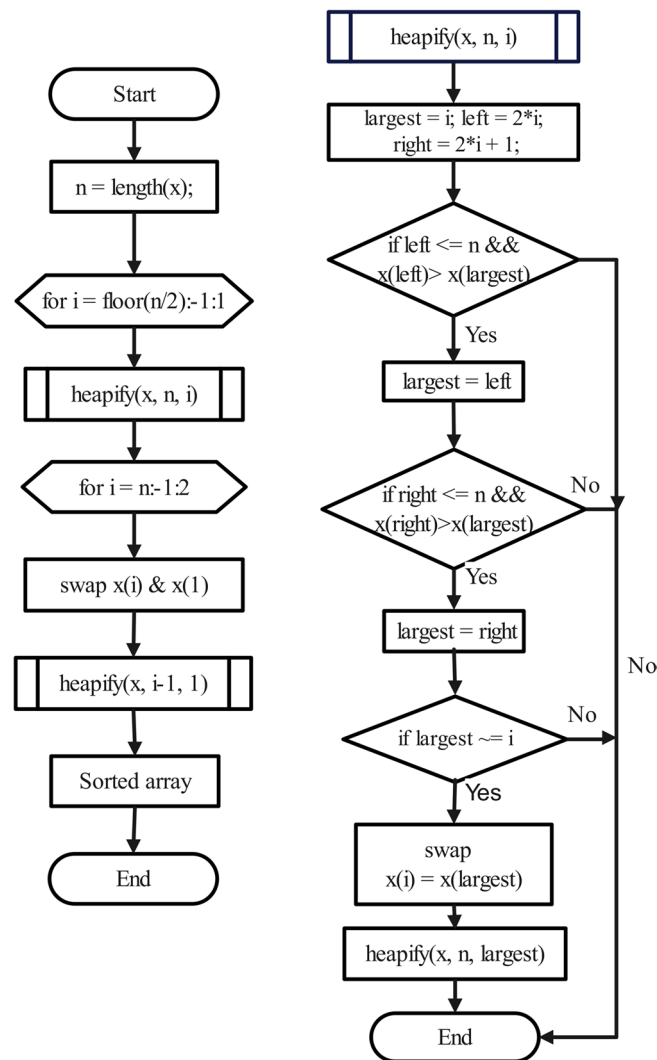


Fig. 12 Flowchart 6.Heap sort algorithm

for integers in base 10, $k = 10$) and perform digit-by-digit sorting for each digit place (starting from the least significant digit) by distribute the elements of the array into the corresponding buckets depends on their current digit value. The elements are collected from the buckets in the exact order in which they are inserted, preserving the order within each bucket. The collected elements are now sorted [67–69]. Figure 20 flowchart describes the operation of radix sort algorithm.

2.11 Selection sort

It is an in-place, simple comparative sorting technique. In this technique, the array is subdivided into unsorted and sorted sub arrays. At first, a sorted sub array is empty and an unsorted array is the initial array. It executes by consistently identifying the smallest one from the unsorted array and placing it in the sorted sub array on the left, while the unsorted elements are placed on the right after every iteration. This procedure continues for the remaining elements in the unsorted sub array unless the whole array is sorted [70–73].The below figure 21 shows steps for selection sort.

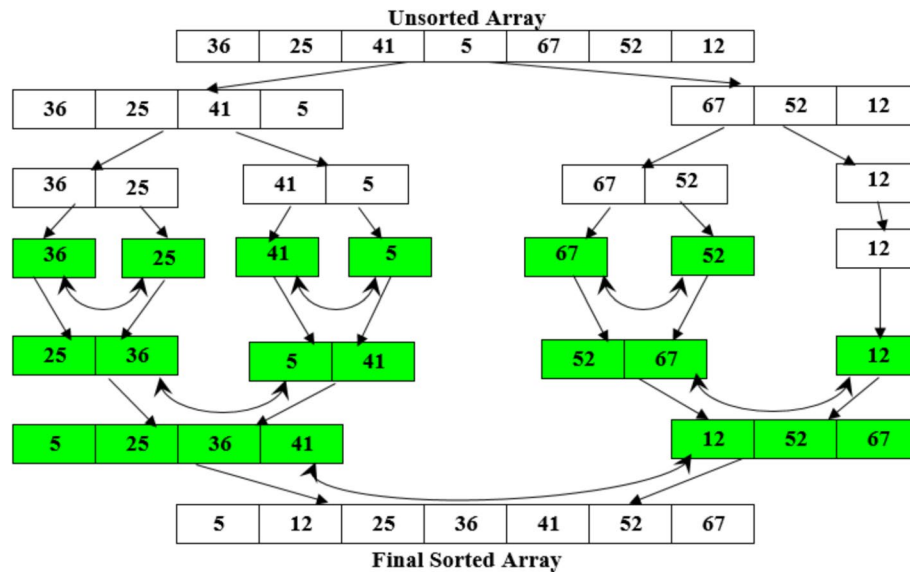


Fig. 13 Example Illustrating Merge Sort Operation

2.11.1 Execution

The algorithm starts with initialize an empty sorted array and an unsorted array containing all the elements to be sorted. It operates by consistently identifying which one in the unsorted array has the lowest value. If the lowest value is identified, then it replaced with the initial unsorted element, putting it in its proper sorted position. The boundary of the sorted side is incremented, considering the just-placed element as part of the sorted section. This process continues iteratively, it gradually increases the sorted section and decreases the unsorted section unless the whole array is sorted [74–76]. Figure 22 flow-chart describes the operation of selection sort algorithm.

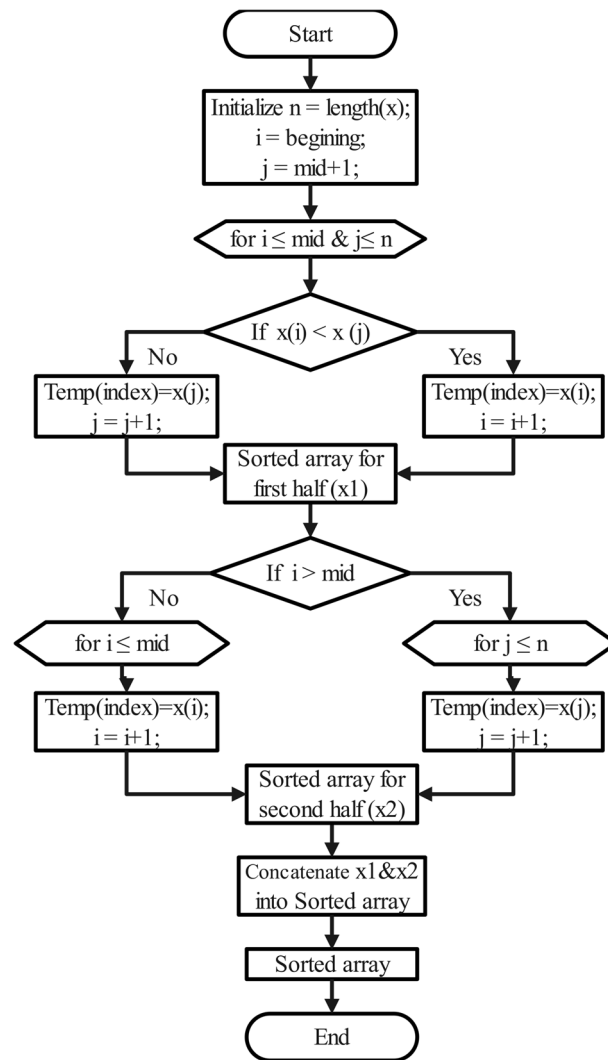
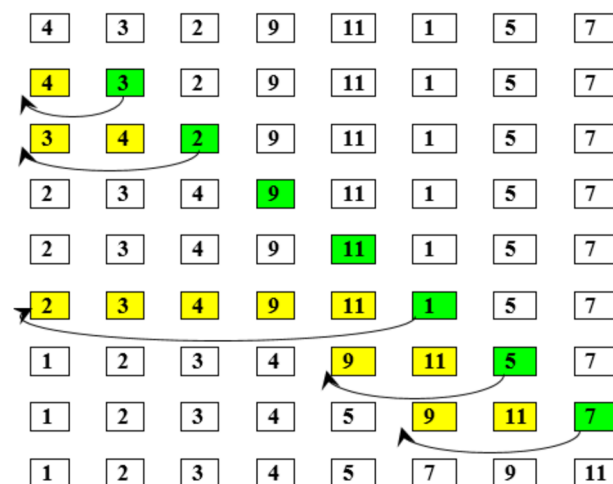
2.12 Shell sort

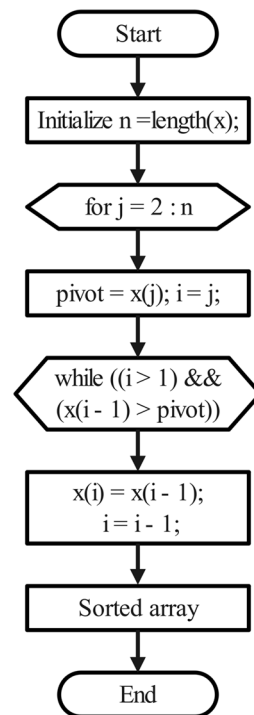
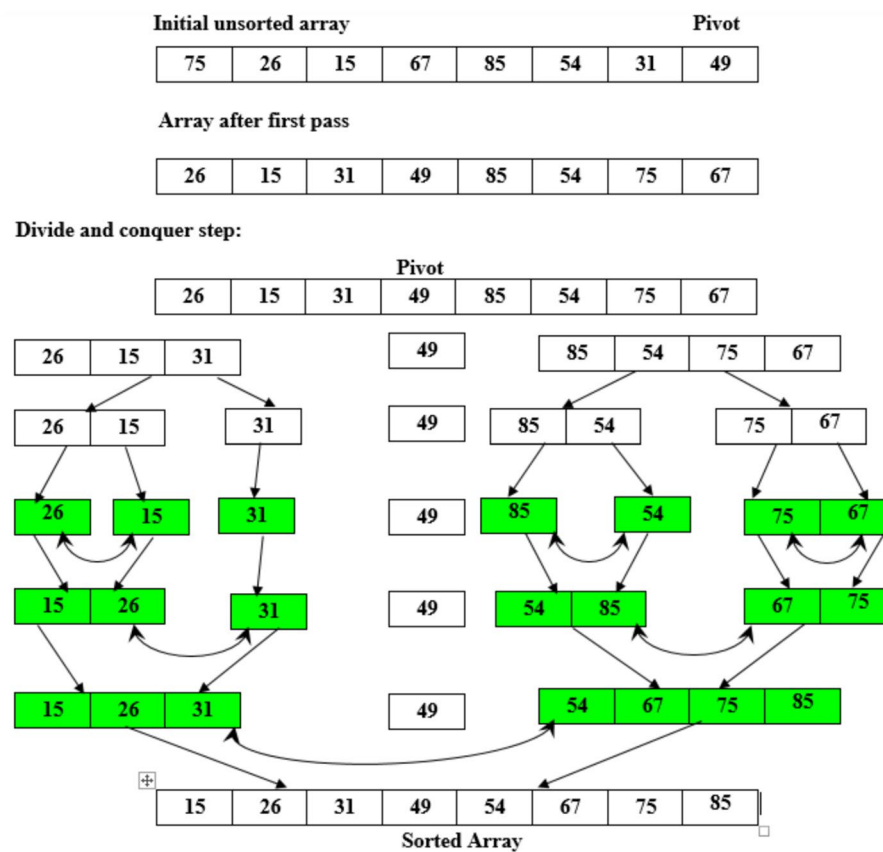
It is an in-place and comparative based sorting technique. It is an advancement over the insertion sort. It operates by sorting the subsets of the entire array and gradually reducing the gap among the elements of the array to be compared and exchanged. At first, it first sorts the elements, which are far from one another, then gradually minimizes the gap size among the elements that are to be compared, unless gap size turns into one, in which case it is effectively an insertion sort. The unique aspect of shell sort is that it tends to move elements faster toward their expected position than a simple insertion sort would, by initially focusing on larger gaps between elements. This leads to a more optimized process when elements are partially sorted [34, 77]. The below figure 23 shows steps for shell sort.

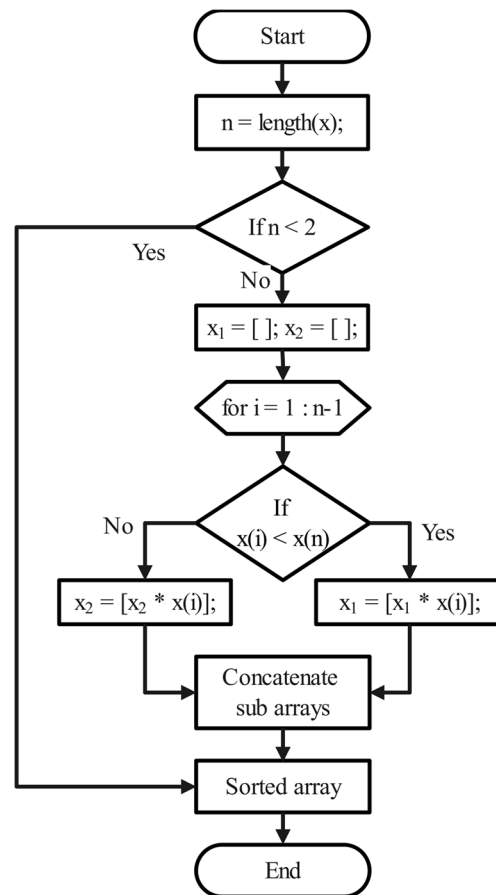
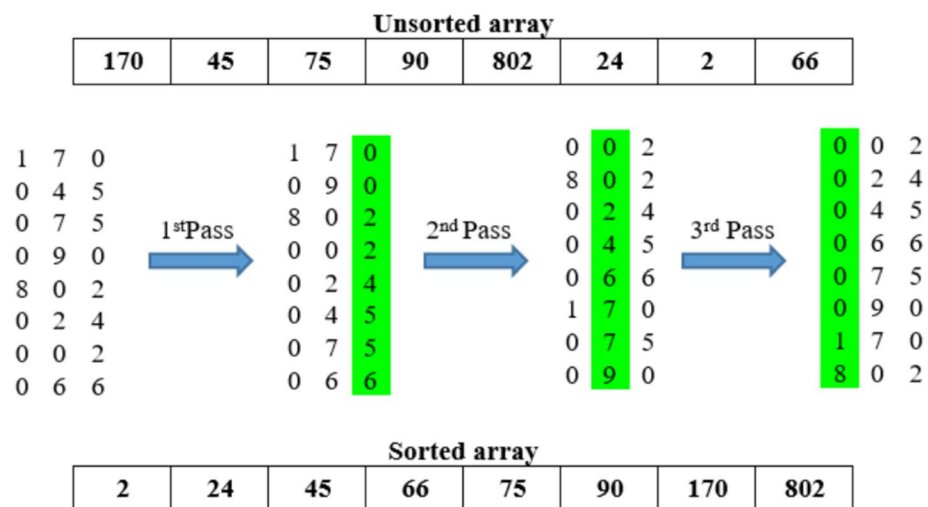
2.12.1 Execution

The execution steps for the shell sort algorithm are as follows:

- Step 1: Initialize the gap: Set the initial gap to half the length of the array ($k=n/2$).
- Step 2: Sort sublists by iterating through the array, comparing each element in the array with elements at the gap distance. When the present element is small, swap the two elements. Repeat this process for all sublists of the array with the same gap distance.

**Fig. 14** Flowchart 7. Merge sort algorithm**Fig. 15** Example Illustrating Insertion Sort Operation

**Fig. 16** Flowchart 8. Insertion sort algorithm**Fig. 17** Example Illustrating Quick Sort Operation

**Fig. 18** Flowchart 9. Quick sort algorithm**Fig. 19** Example Illustrating Radix Sort Operation

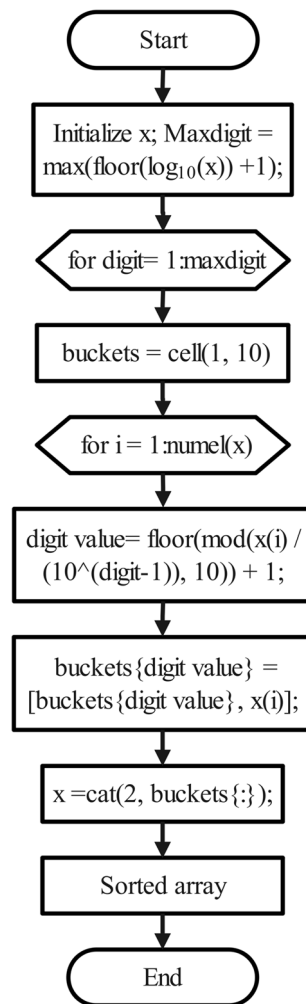


Fig. 20 Flowchart 10. Radix sort algorithm

- Step 3: Reduce the gap by divide the gap by two and repeat steps 2 and 3 until the gap is 1 at which the array is sorted [2, 78]. Figure 24 flowchart describes the operation of shell sort algorithm Table 1 .

3 Complexity analysis of several sorting techniques

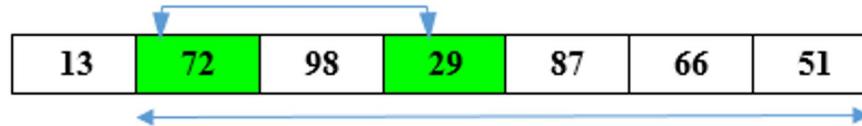
The space and time complexity analysis of sorting techniques is summarized in Table 2. In the best-case, the time complexity of comb, heap, quick, merge and shell sort is $O(n \log n)$. Sorting techniques such as bubble, cocktail and insertion sort have a time complexity of $O(n)$, while non-comparison-based algorithms such as bucket, counting and radix sort achieve a linear time complexity of $O(n + k)$ under favorable input conditions and selection sort have the worst time complexity of $O(n^2)$.

In the average case, the heap, quick, and merge sort maintain a time complexity of $O(n \log n)$. In contrast, bubble, cocktail, insertion, and selection sort have a time complexity of $O(n^2)$. Algorithms that offer better average performance when applied to uniformly distributed or digit-based data, with expected time complexities of $O(n+k)$ for bucket and counting and $O(d(n+k))$ for radix.

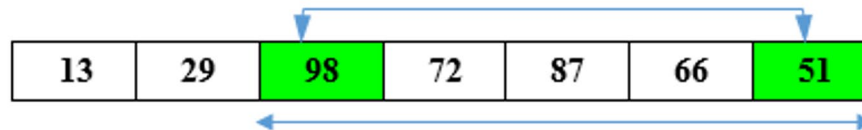
First pass: For first position traverse, 13 is smallest



Second pass: For second position traverse, 29 is smallest



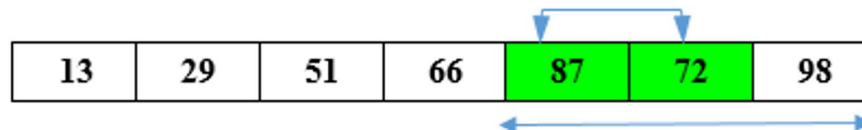
Third pass: For third position traverse, 51 is smallest



Fourth pass: For fourth position traverse, 66 is smallest



Fifth pass: For fifth position traverse, 72 is smallest



Sixth pass: For sixth position traverse, 87 is smallest

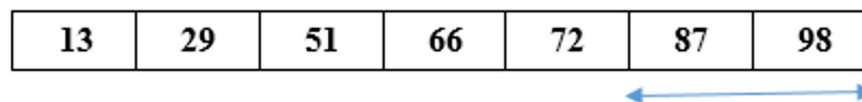


Fig. 21 Example Illustrating Selection Sort Operation

In the worst-case, the heap and merge sort remain efficient with a time complexity of $O(n \log n)$. However, bucket, bubble, comb, cocktail, insertion, quick(with poor pivot choice), selection, and shell sort degrade to a time complexity of $O(n^2)$. Counting and radix sort retain the linear worst-case time complexities of $O(n+k)$ and $O(d(n+k))$, respectively [79–82].

In terms of space complexity, quick sort requires $O(\log n)$ auxiliary space due to its recursive calls. Bubble, cocktail, comb, heap, insertion, shell, and selection sort execute in-place, with a space complexity of $O(1)$. In contrast, bucket and merge sort needs $O(n)$ additional space to store intermediate results, while counting and radix sort require $O(n+k)$ space, where k denotes the range of input values or the number of distinct keys [83–85].

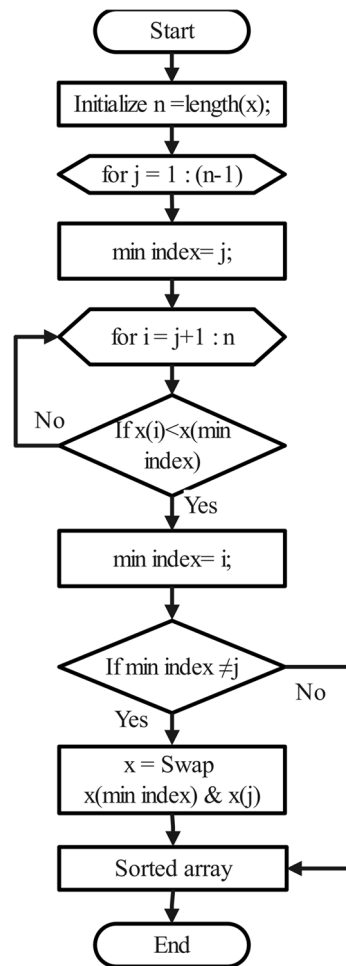


Fig. 22 Flowchart 11.Selection sort algorithm

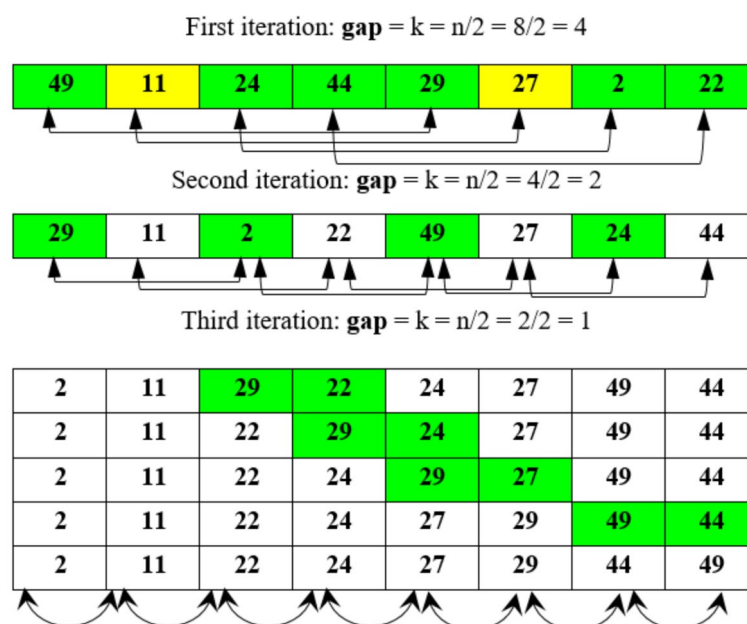


Fig. 23 Example Illustrating Shell Sort Operation

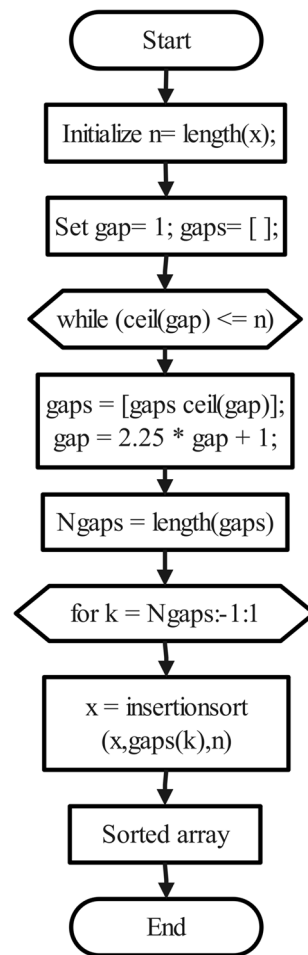


Fig. 24 Flowchart 12.Shell sort algorithm

4 Sorting algorithms execution time comparisons

The various sorting algorithms discussed above are executed by the computer, and its hardware and software specifications are presented in the below table 3.

We have evaluated the performance of above discussed algorithms for the following cases: negative, positive integers, negative, positive decimals, and characters [86, 87]. For each data type, sample arrays were generated using MATLAB's built-in randomization functions. Positive integers were generated using uniformly distributed random values with $\text{randi}([1, 1e5], 1, n)$, while negative integers were produced by negating the same $(-\text{randi}([1, 1e5], 1, n))$. Decimal values (both positive and negative) were generated using $\text{rand}(1, n)$ scaled to the desired range, and for negative decimals, values were negated. Random character arrays were created using ASCII codes with $\text{char}(\text{randi}([65, 90], 1, n))$ for uppercase and $\text{randi}([97, 122], 1, n)$ for lowercase letters. In relation to the volume, the above-mentioned algorithms have been tested in five different cases. In the first case, we tested the algorithm with an array containing 100 numbers/characters; in the second case, we executed the algorithm with an array containing 1000 numbers/characters; in the third case, we tested the algorithm with an array containing 10,000 numbers/characters; in the fourth case, we evaluated the algorithm with an array containing 50,000 numbers/characters; and finally, we tested the algorithm with an array containing 1,00,000 numbers/characters, respectively [88–91].

Table 1 Sorting algorithms: merits, limitations, and typical applications

Sort Tech.	Merits	Limitations	Typical Applications
Bucket	Highly efficient for uniformly distributed data, stable, and easy to parallelize.	Inefficient for large data volumes, not suitable for all data types, and requires extra memory.	Scientific simulations, histogram generation, floating-point data sorting.
Bubble	Easy to understand and implement, requires no additional memory.	Inefficient for large data volumes and nearly sorted datasets.	Sorting small or nearly sorted arrays. Algorithm teaching, visualization.
Cocktail	Bi-directional sorting reduces passes, in-place and easy to implement.	Inefficient for large data volumes, unstable, and not easily parallelizable.	Sorting doubly linked lists.
Comb	Simple, adaptive to nearly sorted data, in-place and effective for small datasets.	Inefficient for large data volumes, unstable, and sensitive to shrink factor.	Used in lightweight environments like JavaScript sorting engines.
Counting	Easy to implement, stable, and efficient for small to medium datasets.	Limited to integer/range-based data, and requires extra memory.	Image processing (e.g., pixel intensity), grading systems, histogram tallying.
Heap	Efficient for large data volumes, in-place sorting, and predictable performance.	More complex to implement, unstable, and has poor cache performance.	Priority queues, job schedulers, streaming median calculations.
Insertion	Simple to implement, stable, in-place, adaptive, and efficient for small datasets.	Inefficient for large data volumes and limited parallelism.	Small arrays, nearly sorted data, embedded systems.
Quick	Fast, adaptive, easy to implement, and efficient for large datasets.	Unstable, performance depends on pivot selection, and may be suboptimal for small datasets.	System-level libraries (e.g., C STL), in-memory data sorting.
Merge	Efficient for large datasets, adaptive, stable, suitable for parallel and external sorting.	Requires extra memory, more complex to implement, and not ideal for small datasets.	External sorting, parallel processing, large-scale databases.
Radix	Efficient for large datasets, stable, non-comparative, and easily parallelized.	Requires more memory, limited to digit-based keys, and inefficient for small datasets.	Sorting long integers, telephone numbers, IP addresses, strings.
Selection	Simple to implement, in-place sorting, and low overhead for very small datasets.	Unstable, not adaptive, and inefficient for large data volumes.	Memory-constrained systems with limited data movement.
Shell	Effective for medium-sized datasets, adaptive, and in-place sorting.	Unstable, performance depends on gap sequence, and not optimal for small datasets.	Real-time systems, firmware environments without recursion support.

Table 2 Space and time complexity of sorting techniques

Sorting Method	Data Type	Space Comp.	Time complexity			Stable
			Worst Case	Average Case	Best Case	
Bucket Sort	Arrays	$O(n)$	$O(n^2)$	$O(n + k)$	$O(n + k)$	✓
Bubble Sort	Arrays	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n)$	✓
Comb Sort	Arrays	$O(1)$	$O(n^2)$	$O(n^2/2^p)$	$O(n \log n)$	×
Cocktail Sort	Arrays	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n)$	✓
Counting Sort	Arrays	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	✓
Heap Sort	Arrays	$O(1)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	×
Insertion Sort	Arrays	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n)$	✓
Quick Sort	Arrays	$O(\log n)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	×
Merge Sort	Arrays	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	✓
Radix Sort	Arrays	$O(n + k)$	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	✓
Shell Sort	Arrays	$O(1)$	$O(n^{3/2})$	$O(n^{4/3})$	$O(n \log n)$	×
Selection Sort	Arrays	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	×

Table 3 Hardware and Software Specifications

Manufacturer name	HP
Model	HP Pro SFF 280 G9 Desktop PC
CPU	12th Generation Intel Core i7–2.10 GHz
RAM	32 Gigabytes DDR4
HDD	1 TB
Operating system	Windows 11 Pro 64 bit
Software	MATLAB R2023a

Table 4 Empirical execution time (in seconds) of sorting techniques on randomly generated positive integers for varying input sizes

Sorting Method	n(e) = 100	n(e) = 1000	n(e) = 10,000	n(e) = 50,000	n(e) = 100,000
Bubble sorting	0.000206	0.005666	0.307565	8.941779	37.716205
Bucket sorting	0.000214	0.000496	0.004663	0.043878	0.094111
Cocktail sorting	0.000201	0.003442	0.302940	8.332764	34.118075
Comb sorting	0.000167	0.000501	0.005212	0.046219	0.104979
Counting sorting	0.000145	0.000399	0.003084	0.033480	0.079580
Heap sorting	0.000182	0.000671	0.007405	0.057028	0.144499
Insertion sorting	0.000167	0.000774	0.041888	0.982159	3.881153
Quick sorting	0.000196	0.000550	0.005191	0.042325	0.094456
Merge sorting	0.000193	0.000581	0.005451	0.043847	0.095689
Radix sorting	0.000378	0.001912	0.016153	0.097779	0.198698
Selection sorting	0.000166	0.001111	0.068992	1.622368	6.417697
Shell sorting	0.000170	0.000485	0.004752	0.043383	0.096593

Table 5 Empirical execution time (in seconds) of sorting techniques on randomly generated negative integers for varying input sizes

Methods of sort	n(e) = 100	n(e) = 1000	n(e) = 10000	n(e) = 50000	n(e) = 100000
Bubble sorting	0.000185	0.003794	0.317751	9.570930	39.805249
Bucket sorting	0.000194	0.000478	0.004705	0.042633	0.094542
Cocktail sorting	0.000194	0.003707	0.314089	8.434392	34.272231
Comb sorting	0.000158	0.000498	0.005533	0.044706	0.098111
Heap sorting	0.000182	0.000660	0.007585	0.059279	0.143672
Insertion sorting	0.000167	0.000768	0.042206	0.972823	3.799231
Quick sorting	0.000169	0.000461	0.005120	0.045748	0.102123
Merge sorting	0.000182	0.000416	0.005514	0.041782	0.094571
Radix sorting	0.000395	0.001877	0.016759	0.101023	0.213184
Selection sorting	0.000179	0.001121	0.068659	1.627446	6.444718
Shell sorting	0.000180	0.000488	0.005191	0.044445	0.159015

Table 4 shows that, when the algorithms are executed with randomly generated positive integers, bubble sort has the worst execution time for all volumes of positive integer numbers except radix sort, which performs worst for small data volume of $n = 100$, while the counting sort has the best execution time for all volumes of positive integer numbers. The following table shows the results for the case in which the array's elements are randomly chosen negative integers with execution time measured in seconds[92–94].

The execution times for sorting techniques with randomly chosen negative integers are presented in table 5, bubble sort has the worst execution time for all volumes of negative integer numbers except radix sort, which performs worst for the case $n = 100$, while the bucket sort has the best execution time for all volumes of negative integer numbers

Table 6 Empirical execution time (in seconds) of sorting techniques on randomly generated positive decimal numbers for varying input sizes

Methods of sort	n(e) = 100	n(e) = 1000	n(e) = 10000	n(e) = 50000	n(e)=100000
Bubble sorting	0.000230	0.003865	0.319457	9.680891	39.573066
Bucket sorting	0.000234	0.000702	0.005663	0.033600	0.100145
Cocktail sorting	0.000246	0.003790	0.298496	8.209032	33.427447
Comb sorting	0.000202	0.000730	0.006660	0.040996	0.086286
Heap sorting	0.000213	0.000904	0.008838	0.051978	0.128068
Insertion sorting	0.000192	0.000997	0.043630	0.962210	3.841356
Quick sorting	0.000219	0.000715	0.006100	0.035586	0.079548
Merge sorting	0.000221	0.000798	0.006668	0.034727	0.079085
Radix sorting	0.000423	0.002196	0.017215	0.086667	0.182345
Selection sorting	0.000206	0.001404	0.069889	1.621919	6.396352
Shell sorting	0.000204	0.000705	0.006278	0.036201	0.082412

Table 7 Empirical execution time (in seconds) of sorting techniques on randomly generated negative decimal numbers for varying input sizes

Methods of sort	n(e) = 100	n(e) = 1000	n(e) = 10000	n(e) = 50000	n(e)= 100000
Bubble sorting	0.000231	0.003953	0.307105	9.038311	38.222265
Bucket sorting	0.000240	0.000693	0.005801	0.035743	0.083559
Cocktail sorting	0.000219	0.003882	0.296157	8.216716	33.328224
Comb sorting	0.000195	0.000724	0.006568	0.047399	0.111019
Heap sorting	0.000227	0.000896	0.009068	0.063983	0.200014
Insertion sorting	0.000205	0.001038	0.043724	0.999167	3.873828
Quick sorting	0.000208	0.000717	0.006662	0.060964	0.172023
Merge sorting	0.000220	0.000798	0.006966	0.064204	0.178478
Radix sorting	0.000416	0.002165	0.017125	0.128487	0.256303
Selection sorting	0.000214	0.001338	0.070763	1.669013	6.526018
Shell sorting	0.000210	0.000699	0.006651	0.062801	0.176315

except comb and merge sort, which performs best for small data volume of $n = 100$ and 1000 [95, 96].

Table 6 presents the results for randomly generated positive decimal numbers, bubble sort has the worst execution time for all volumes of positive decimal numbers except radix sort, which performs worst for the case $n = 100$, while the insertion sort performs best execution time for small data volume of $n=100$ and the merge sort performs best for large data volume of $n=100000$, for all other cases bucket sort performs best [97, 98].

In table 7 shows that when the algorithms are executed with randomly generated negative decimal numbers, bubble sort has the worst execution time for all volumes of negative decimal numbers except radix sort, which performs worst for the case $n = 100$, while the bucket sort performs best execution time for all volumes of data, except comb sort, which performs best for small data volume of $n = 100$ [99, 100].

The results for sorting techniques on randomly generated character arrays are shown in table 8, bubble sort has the worst execution time for all volumes of negative integer numbers except radix sort, which performs worst for the case $n = 100$, while the bucket sort performs best execution time for all volumes of data, except comb sort, which performs best for small data volume of $n = 100$ [101, 102].

Table 8 Empirical execution time (in seconds) of sorting techniques on randomly generated characters for varying input sizes

Methods of sort	n(e) = 100	n(e) = 1000	n(e) = 10000	n(e) = 50000	n(e)= 100000
Bubble sorting	0.000179	0.003044	0.295605	8.795471	36.716840
Bucket sorting	0.000156	0.000171	0.000541	0.001663	0.003954
Cocktail sorting	0.000182	0.003041	0.287000	8.164508	33.444204
Comb sorting	0.000135	0.000200	0.000895	0.004331	0.010937
Heap sorting	0.000152	0.000420	0.003882	0.023106	0.047830
Insertion sorting	0.000133	0.000482	0.033946	0.820740	3.280556
Quick sorting	0.000155	0.000253	0.003059	0.045940	0.162975
Merge sorting	0.000164	0.000322	0.001321	0.006144	0.011228
Radix sorting	0.000353	0.001546	0.012651	0.059902	0.120988
Selection sorting	0.000140	0.000774	0.058723	1.425416	5.720006
Shell sorting	0.000162	0.000206	0.000743	0.003046	0.006191

5 Future directions

Even though this analysis provides a thorough empirical assessment of various classical sorting techniques for various data types and sizes, but still plenty of aspects that might be explored further: One area of advancement is to develop the hybrid sorting techniques that merge the strengths of existing methods to handle varied data distributions more efficiently. Further, implementing sorting techniques in parallel and distributed computing platforms such as multi-core CPUs, GPUs, TPUs, or big data platforms like Spark can significantly improve scalability and performance for real-time and large-scale datasets.

In additional hardware optimisation, such as branch prediction and cache-efficient memory access, may enhance execution performance, especially for divide-and-conquer algorithms like quick sort and merge sort. Another significant area is the integration of machine learning models to dynamically opting the optimal sorting technique based on input characteristics, enabling intelligent and adaptive systems.

6 Conclusion

Sorting the data is one of the most critical tasks in many real-time applications, and optimizing sorting techniques remains an essential area of research. In this study, we analyzed and executed several classical sorting algorithms using the MATLAB platform, evaluating their time and space complexities as well as their practical execution times across different data types and input volumes. The algorithms were tested on positive and negative integers, positive and negative decimal numbers, and character arrays, with input sizes ranging from 100 to 100,000 elements. In the context of time complexity, we executed several sorting techniques with various data types for different volumes, we noticed that:

- In the case of randomly chosen positive integers, the counting sort is most efficient and fastest algorithm to implement, whereas bubble sorting is slowest.
- For the cases of randomly chosen negative integers, the bucket sort is the most efficient and fastest algorithm to implement, and the bubble sorting is slowest, whereas counting sorting functions in a MATLAB platform does not support the negative integers.
- In the case of random positive and negative decimal numbers, the bucket sort is the most efficient and fastest algorithm to implement, and the bubble sorting is slowest,

whereas counting sorting functions in a MATLAB platform does not support the decimal numbers.

- For the case of random characters, the bucket sort is the most efficient and fastest algorithm to implement, and the bubble sorting is slowest, whereas counting sorting functions in a MATLAB platform does not support the characters.

Our findings show that specific sorting algorithms are executed more efficiently for smaller data quantities, while others are executed inefficiently for larger data quantities. Certain sorting algorithms perform better with huge quantities of data, while others perform worse with smaller quantities of data. In terms of space complexity, quick sort have $O(\log(n))$, where as cocktail, comb, bubble, heap, insertion, selection, and shell sorting have $O(1)$ space complexity; on the other hand, bucket, counting, merge and radix sorting have $O(n)$ space complexity.

Although the theoretical analysis (as shown in table 2) identifies merge and heap sort with $O(n \log(n))$ time complexity as highly efficient, our empirical results show that bucket sort performs better for data sizes up to 100,000 elements. In the bucket sort, when we applied a uniformly distributed data, it incurs minimal overhead and demonstrates excellent runtime performance for moderate data sizes. However, as the data size increases beyond 100,000 elements, the bucket sort performance begins to degrade due to increased bucket management and memory usage, allowing the merge and heap sort to regain their advantage through more scalable and stable performance.

Acknowledgements

This work was supported by the School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, Tamil Nadu, India.

Author contributions

Sureshkumar Sundaramoorthy and Govardhan Karunanidhi are equally contributed for research investigation, conceptualization, analysis, methodology, software, validation, and manuscript draft, and review.

Funding

Open access funding provided by Vellore Institute of Technology. No funding was received for conducting this study.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval

Not applicable.

Consent for Publication

Not applicable.

Competing interests

The authors declare no Conflict of interest.

Received: 26 March 2025 / Accepted: 8 September 2025

Published online: 03 November 2025

References

1. Armend S, Hoti MH, Hoti. A. H. A Review of Performance and Complexity on Sorting Algorithms. 2022 International Conference on Computing, Networking, Telecommunications and Engineering Sciences Applications (CoNTESA) 2022;45–50 <https://doi.org/10.1109/contesa57046.2022.10011382>
2. Cormen TH, Charles ELRRL, Stein C. Introduction to algorithms. (The Mit Press, 2022).
3. Jehad A, Rami M. An enhancement of major sorting algorithms. *Int Arab J Inf Technol*. 2010;07:55–62.
4. Mishra AD, Garg D. Selection of best sorting algorithm. *Int J Intell Inf Process*. 2008;2:363–8.
5. Kumar S. An empirical study on algorithms - conquer sorting techniques. *SSRN Electron J*. 2020. <https://doi.org/10.2139/ssrn.3568396>.
6. AL-Azzam S, Qatawneh M. Parallel processing of sorting and searching algorithms comparative study. *Mod Appl Sci*. 2018;12:143.

7. Akhter N, Idrees M, Furqan-ur-Rehman. Sorting algorithms - a comparative study. *Int J Comput Sci Inf Secur*. 2016;14:930–6.
8. Cook CR, Kim DJ. Best sorting algorithm for nearly sorted lists. *Commun ACM*. 1980;23:620–4.
9. Zutshi A, Goswami D. Systematic review and exploration of new avenues for sorting algorithm. *Int J Inf Manag Data Insights*. 2021;1:100042.
10. Rajagopal D, Thilakavalli K. Comparison of different sorting algorithms in data structure based upon the time complexity. *Int J Technol*. 2016;6:23.
11. Karimizadeh MM, Rafeazadeh E, Amiri P, Kholghnik D. Enhancing and optimization sorting algorithms: an empirical study. *Cumhuriyet Sci J*. 2015;36:254–66.
12. Setiawan R. Comparing sorting algorithm complexity based on control flow structure. *Int Conf Inf Manag Technol*. 2016;2016:224–8. <https://doi.org/10.1109/icimtech.2016.7930334>.
13. Ali, et al. Comparison of different sorting algorithm. *Int J Adv Res Comput Sci Electron Eng*. 2016;5:63–71.
14. Aliyu AM, Zirra PB. A comparative analysis of sorting algorithms on integer and character arrays. *Int J Eng Sci (IJES)*. 2013;2:25–30.
15. Kaur S. Study of sorting algorithm to optimize search results. *Int J Emerg Trends Technol Comput Sci*. 2013;2:204–7.
16. Al-Kharabsheh KS, AlTurani IM, AlTurani AMI, Zanoon NI. Review on sorting algorithms a comparative study. *Int J Comput Sci Secur*. 2013;7:120–6.
17. Peng Q. Bubble Sort and Its Improved Methods. 2023 IEEE 3rd International Conference on Data Science and Computer Application (ICDSCA) 2023;367–370 <https://doi.org/10.1109/icdscas59871.2023.10393066>
18. Ganapathi P, Chowdhury R. Parallel divide-and-conquer algorithms for bubble sort, selection sort and insertion sort. *Comput J*. 2021;65:2709–19.
19. Rahmani KI, M. Smart bubble sort: a novel and dynamic variant of bubble sort algorithm. *Comput Mater Continua*. 2022;71:4895–913.
20. Aqib SM, Nawaz H, Butt SM. Analysis of merge sort and bubble sort in python, php, javascript, and c language. *Int J Adv Trends Comput Sci Eng*. 2021;10:680–6.
21. Edjlal R, Edjlal A, Moradi T. A sort implementation comparing with Bubble sort and Selection sort. *IEEE Xplore* 2011;vol. 4 380–381 <https://ieeexplore.ieee.org/document/5763927>.
22. Min W. Analysis on Bubble Sort Algorithm Optimization. *IEEE Xplore* 2010;vol. 1 208–211 <https://ieeexplore.ieee.org/abstract/document/5635119?casa-token=yRpXS-rtAjqAAAA:lLOKuCmVsKJZdiEAFg5P-6YK-DONx29E-M6-SFypPzroT9eHrYpBWOLi-Ativpx6fPZMSF6chE>.
23. Rohil H, sha, M. Run time bubble sort - an enhancement of bubble sort. *Int J Comput Trends Technol*. 2014;14:36–8.
24. Arora N, Nirmal J, Tamta VK. A novel sorting algorithm and comparison with bubble sort and insertion sort. *Int J Comput Appl*. 2012;45:31–2.
25. Agha FA, Nawaz H. Comparison of bubble and insertion sort in rust and python language. *Int J Adv Trends Comput Sci Eng*. 2021;10:1020–5.
26. Kumar S, Singla P. Sorting using a combination of bubble sort, selection sort and counting sort. *Int J Math Sci Comput*. 2019;5:30–43.
27. Paul T. Enhancement of Bubble and Insertion Sort Algorithm Using Block Partitioning. 25th International Conference on Computer and Information Technology IEEE 2022;412–417 <https://doi.org/10.1109/iccit57492.2022.10055404>
28. Saadeh R, Qatawneh M. Performance Evaluation of Parallel Bubble Sort Algorithm on Supercomputer IMAN1. *Int J Comput Sci Inf Technol*. 2019;11:2.
29. Yang L. The array-based bucket sort algorithm. *J Comput Res Dev*. 2007;44:341.
30. Khurana M, Neetu F, Shipra S. Hybrid bucket sort switching internal sorting based on the data inside the bucket. International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO) 2017;476–482 <https://doi.org/10.1109/icrito.2017.8342474>
31. Fenyi A, Fosu M, Appiah B. Comparative analysis of comparison and non comparison based sorting algorithms. *Int J Comput Appl*. 2020;175:22–5.
32. Burnetas A, Solow D, Agarwal R. An analysis and implementation of an efficient in-place bucket sort. *Acta Inf*. 1997;34:687–700.
33. Neetu F, Shipra S. The detailed experimental analysis of bucket sort. In 2017 7th International Conference on Cloud Computing, Data Science and Engineering-Confluence 2017;1–6 <https://doi.org/10.1109/confluence.2017.7943114>
34. Goel A, Sharma A, Gupta N. A variant of Bucket Sort Shell Sort vs Insertion Sort. International Conference on Computing, Communication and Networking Technologies (ICCCNT) 2019;1–5 <https://doi.org/10.1109/icccnt45670.2019.8944607>
35. Horsmalahiti P. Comparison of Bucket Sort and Radix Sort. *CoRR abs/1206.3511*, 2012;.
36. Komalasari c, Istiono. W. A., Comparative Study of Cocktail Sort and Insertion Sort. *J Appl Comput Sci Math*. 2021;15:21–5.
37. Elkahilout AH, Ashraf. A comparative Study of Sorting Algorithms Comb, Cocktail and Counting Sorting. *Int Res J Eng Technol(IJRET)*. 2017;4:1387–90.
38. Sun W, Ma Z. Count sort for gpu computing. International Conference on Parallel and Distributed Systems. 2009;919–924. DOI: <https://doi.org/10.1109/icpads.2009.30>.
39. Bajpai K, Kots A. Implementing and analyzing an efficient version of counting sort (e-counting sort). *Int J Comput Appl*. 2014;98:1–2.
40. Usmani AR. A Novel Time and Space Complexity Efficient Variant of Counting-Sort Algorithm. *IEEE Xplore* 2019;1–6 <https://ieeexplore.ieee.org/abstract/document/8966717https://doi.org/10.1109/ICIC48496.2019.8966717>
41. Kumar S, Madhu SG, Kumar K, Prem S.S. A Novel Counting Sort for Real Numbers with Linear Time Complexity. 2022 International Conference on Computational Intelligence and Sustainable Engineering Solutions (CISES) 2022;60–64 <https://doi.org/10.1109/cises54857.2022.9844325>
42. Li H, Chen P, Wang Y. Heap sorting based on array sorting. *J Comput Commun*. 2017;05:57–62.
43. Ali H, Nawaz H, Maitlo A, Soomro I. Performance analysis of heap sort and insertion sort algorithm. *Int J Emerg Trends Eng Res*. 2021;9:580–6.
44. Marcellino M, Pratama DW, Suntiarko SS, Margi K. Comparative of Advanced Sorting Algorithms (Quick Sort, Heap Sort, Merge Sort, Intro Sort, Radix Sort) Based on Time and Memory Usage. *IEEE Xplore* 2021;vol. 1 154–160 <https://ieeexplore.ieee.org/abstract/document/9609715>.

45. Sharma V, Sandhu PS, Singh S, Saini B. Analysis of modified heap sort algorithm on different environment. *Int J Electr Comput Eng*. 2008;2:1143–5.
46. Zhenhua W, Zhifeng L, Guoliang L. Parallel Optimization Strategy of Heap Sort Algorithm under Multi-core Environment. *IEEE Xplore* 2015;768–771 <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&number=7263684><https://doi.org/10.1109/ICMTMA.2015.190>
47. Taiwo OE, Christianah AO, Oluwatobi, AN, Aderonke KA, others. Comparative study of two divide and conquer sorting algorithms: quicksort and mergesort. *Procedia Computer Science* 2020;171, 2532–2540.
48. Vignesh R, Pradhan T. Merge sort enhanced in place sorting algorithm. in 2016 International Conference on Advanced Communication Control and Computing Technologies (ICACCT) 2016;698–704
49. Mishal I, AL-Khatib R, Hiasat R. Comparative Study of Two Divide and Conquer Sorting Algorithms: Modified Quick Sort and Merge Sort. *Int J Comput Appl*. 2021;183:28–33.
50. Lobo J, Kuwelkar S. Performance analysis of merge sort algorithm. in 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC) 2020;110–115 (IEEE).
51. Bai Y, Guo X. Efficient algorithm of depth-first stable in-place merge sort. *J Comput Appl*. 2013;33:1039–42.
52. Uyar A. Parallel merge sort with double merging. *IEEE Xplore* 1–5 <https://ieeexplore.ieee.org/document/7036012/citations?tabFilter=papers#citations> 2014; <https://doi.org/10.1109/ICAICT.2014.7036012>
53. Panny W, Prodinger H. Bottom-up mergesort-a detailed analysis *Algorithmica* Springer. 1995;14:340–54. <https://doi.org/10.1109/pdp.2007.10>.
54. Manwade KB. Analysis of parallel merge sort algorithm. *Int J Comput Appl*. 2010;1:70–3.
55. Woźniak M, Marszałek Z, Gabryel M, Nowicki, R. Modified merge sort algorithm for large scale data sets. *Artificial Intelligence and Soft Computing: 12th International Conference, ICAISC, Zakopane, Poland, June 9–13, 2013, proceedings, part II* 12. Springer. 2013;2013:612–22.
56. Marszałek Z. Performance tests on merge sort and recursive merge sort for big data processing. *Tech Sci*. 2017;21:19–35.
57. Peeters J, Haase J. Improving mergesort performance by optimizing branch prediction. *IEEE Asia-Pacific Conference on Computer Science and Data Engineering*. 2022. <https://doi.org/10.1109/csde56538.2022.10089293>.
58. Renu R, Manisha M. Mq sort an innovative algorithm using quick sort and merge sort. *Int J Comput Appl*. 2015;122:10–4.
59. Jiang D, Zhou MA. comparative study of insertion sorting algorithm verification. *IEEE 2nd information technology, networking, Electronic and Automation Control Conference (ITNEC)*. 2017;2017. DOI: <https://doi.org/10.1109/itnec.2017.8284998>.
60. SinghSodhi T, Kaur S, Kaur S. Enhanced insertion sort algorithm. *Int J Comput Appl*. 2013;64:35–9.
61. Biernacki C, Jacques J. A generative model for rank data based on insertion sort algorithm. *Comput Stat Data Anal*. 2013;58:162–76.
62. Kalaivani A, Swetha K. An enhanced bidirectional insertion sort over classical insertion sort. *Int J Image Graph*. 2020;21:2150024.
63. Mohammed AS, Amrahov ŞE, Şelebi FV. Bidirectional conditional insertion sort algorithm; an efficient progress on the classical insertion sort. *Futur Gener Comput Syst*. 2017;71:102–12.
64. Rajput IS, Suresh Kumar B, Singh T. Performance comparison of sequential quick sort and parallel quick sort algorithms. *Int J Comput Appl*. 2012;57:14–22.
65. Frias L, Roura S. Multikey quickselect *Algorithmica* Springer. 2014;69:958–73. <https://doi.org/10.1109/csde56538.2022.10089318>.
66. Fouz M, Kufleitner M, Manthey B, Zeini JN. On smoothed analysis of quicksort and hoare's find. *Algorithmica* Springer. 2012;62:879–905.
67. Manjunath D. T-sort: a tagging technique to optimize memory utilization in radix sort. *IOSR J Comput Eng*. 2012;7:30–5.
68. NG W, KAKEHI K. Cache Efficient Radix Sort for String Sorting. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences E90-A*, 2007;457–466.
69. Shukla A. Modified pure radix sort for large heterogeneous data set. *IOSR J Comput Eng*. 2012;3:20–3.
70. Hayfron-Acquah BJ, Appiah O, Riverson K. Improved selection sort algorithm. *Int J Comput Appl*. 2015;110:29–33.
71. Vilchez RN. Bidirectional enhanced selection sort algorithm technique. *Int J Appl Phys Sci*. 2019;5:1–7.
72. Vilchez RN. Modified selection sort algorithm employing boolean and distinct function in a bidirectional enhanced selection technique. *Int J Mach Learn Comput*. 2020;10:93–8.
73. Bilateral ZA, Sort S. International Journal for Research in Applied Science and Engineering Technology. 2020;8:230–6.
74. He D. Some Improvements in Selection Sorting. In 2023 IEEE 3rd International Conference on Data Science and Computer Application (ICDSCA) 2023;371–374 <https://doi.org/10.1109/icdscas9871.2023.10393273>
75. Naz A, Nawaz H, Maitlo A, Hassan SM. Implementation of selection sort algorithm in various programming languages. *Int J Adv Trends Comput Sci Eng*. 2021;10:2249–55.
76. Rabiun AM, Garba EJ, Baha BY, Mukhtar MI. Comparative analysis between selection sort and merge sort algorithms. *Nigerian J Basic Appl Sci*. 2022;29:43–8.
77. Stochastic analysis of shell sort. Smythe, Robert T and Wellner. *J Algorithmica* Springer. 2001;31:442–57.
78. Radavičius I, Baranauskas M. An empirical study of the gap sequences for shell sort. *Lietuvos matematikos rinkinys*. 2013;54:61–6.
79. Yerram B, Bhonagiri JK. An Efficient Sorting Algorithm for binary data. *IEEE Xplore* 1–4 <https://ieeexplore.ieee.org/abstract/document/9225518> 2020; <https://doi.org/10.1109/ICCCNT49239.2020.9225518>
80. Purvi P, Nikita B, Nirav B. Performance Comparison of Different Sorting Algorithms. *International Journal of Latest Technology in Engineering, Management and Applied Science (IJTEMAS)* VI, 2017;39–41.
81. Santos BG, et al. Asymptotic Analysis of the Running Time Performed by Various Sorting Algorithms. 2021 International Conference on Intelligent Technologies (CONIT) 2021; <https://doi.org/10.1109/conit51480.2021.9498490>
82. Kumar M, Malhotra MsM, Ahuja MsD. Minimising the Execution Time of Selection Sort Algorithm. *International Journal of Engineering And Computer Science* 2015;6, 21.
83. Pizarro-Vasquez GO, Fabiola MM, Pierina GM, Botto-Tobar M. Sorting Algorithms and Their Execution Times an Empirical Evaluation. *Adv Intell Syst Comput*. 2020. <https://doi.org/10.1007/978-3-030-63665-4-27>.
84. Raheja S, Kukreja V. Enhancements in sorting algorithms: a review, journal of today's ideas - tomorrow's technologies 2015;3:73–82.

85. Shabbir A, et al. A Review of Algorithms's Complexities on Different Valued Sorted and Unsorted Data. *International Conference on IT and Industrial Technologies (ICIT)* 2023;1–6. <https://doi.org/10.1109/icit59216.2023.10335840>
86. Bharadwaj A, Mishra S. Comparison of sorting algorithms based on input sequences. *Int J Comput Appl*. 2013;78:7–10.
87. Buradagunta S, Bodapati JD, Mundukur NB, Salma S. Performance comparison of sorting algorithms with random numbers as inputs. *Ingénierie des systèmes d'information*. 2020;25:113–7.
88. Kumari A, Singh NK, Chakraborty S. A statistical comparative study of some sorting algorithms. *Int J Found Comput Sci Technol*. 2015;5:21–9.
89. Edelkamp S, Weiß A, Wild S. Quicksort: a fast sorting scheme in theory and practice. *Algorithmica* Springer. 2020;8:509–88.
90. Durad MH, Akhtar MN, Irfan-ul-Haq. Performance Analysis of Parallel Sorting Algorithms Using MPI. *IEEE Xplore*. 2014;202–207. <https://ieeexplore.ieee.org/document/7118400https://doi.org/10.1109/FIT.2014.46>
91. Suresh A, George AK. Performance analysis of various combination sorting algorithms for large dataset to fit to a multi-core architecture. in 2018 second international conference on inventive communication and computational technologies (ICICCT) 2018;51–56.
92. Durrani, OK, Abdulhayan, S. Performance Measurement of Popular Sorting Algorithms Implemented using Java and Python. in 2022 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICEC-CME). 2022;1–6.
93. Durrani OK, Farooqi AS, Chinmai AG, Prasad KS. Performances of Sorting Algorithms in Popular Programming Languages. *IEEE Xplore*. 2022;1–7 <https://ieeexplore.ieee.org/abstract/document/10084261https://doi.org/10.1109/SMARTGENCONS6628.2022.10084261>
94. Faujdar N, Ghrera SP. Analysis and Testing of Sorting Algorithms on a Standard Dataset. *IEEE Xplore* 962–967 <https://ieeexplore.ieee.org/abstract/document/7280062/> (2015) <https://doi.org/10.1109/CSNT.2015.98>
95. Hoda A, Omar Y, Badr. A. Mapping sorting algorithm. 2016 SAI Computing Conference (SAI) 2016; <https://doi.org/10.1109/sai.2016.7556025>
96. Lakshmi Dr I. Performance Analysis of Four Different Types of Sorting Algorithms using Different Languages. *Int J Trend Sci Res Dev*. 2018;2:535–41.
97. Dawood AS, Abburi KS. BQM Hybrid Sorting Algorithm. *International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*. 2023;1–5. <https://doi.org/10.1109/csitts60515.2023.10334130>
98. Han Y. Sorting real numbers in $O(n \log n)$ time and linear space. *Algorithmica*, Springer. 2020;82 966–978.
99. Pinnamaraju C. Binary Search Sort Algorithm -Yet Another Sorting Algorithm with Binary Search having $O(n \log n)$ and $O(n)$ time complexity. 2023 1st International Conference on Optimization Techniques for Learning (ICOTL) 2023;1–6 <https://doi.org/10.1109/icotl59758.2023.10435020>
100. Goel K, Dwivedi P, Sharma O. Performance Analysis of Various Sorting Algorithms: Comparison and Optimization. 2023 11th International Conference on Intelligent Systems and Embedded Design (ISED) 2023;1–5 <https://doi.org/10.1109/ised59382.2023.10444609>.
101. Dutta A, et al. Computational Time Complexity for Sorting Algorithm amalgamated with Quantum Search. 2023 International Conference for Advancement in Technology (ICONAT) 1–6 (2023) <https://doi.org/10.1109/iconat57137.2023.10080217>
102. Bingmann T, Eberle A, Sanders P. Engineering parallel string sorting, *Algorithmica*, Springer 235–286 (2017)
103. Brian M. Sorting Methods. www.mathworks.com
104. Singla S, Mangla D, Rani S, Modibbo, UM. Impact of preventive maintenance and failure rate on a complexly configured system: A sensitive analysis. *Reliability: Theory & Applications* 2024;19(4(80)), 774–791.
105. Roşca, C-M, Cărbureanu, M. A Comparative Analysis of Sorting Algorithms for Large-Scale Data: Performance Metrics and Language Efficiency. *International Conference on Emerging Trends and Technologies on Intelligent Systems*, Singapore: Springer Nature Singapore, 2024 ; pp. 99–113.
106. Abuba NS, Baagyere EY, Nakpih CI, Wiredu JK. Optiflexsort: a hybrid sorting algorithm for efficient large-scale data processing. *J Adv Math Comput Sci*. 2025;40(2):67–81.
107. Rizvi, QM, Rai H, Jaiswal R. Sorting Algorithms in Focus: A Critical Examination of Sorting Algorithm Performance. *Emerging Trends in IoT and Computing Technologies*, 2024;103–106.
108. Sabah AS, Abu-Naser, SS, Helles YE, Abdallatif RF, Samra, FYAA, Taha AHA, Massa, NM, amouda, AA. Comparative analysis of the performance of popular sorting algorithms on datasets of different sizes and characteristics. 2023;
109. Zutshi A, Goswami D. Systematic review and exploration of new avenues for sorting algorithm. *Int J Inf Manag Data Insights Elsevier*. 2021;1(2):100042.
110. Chauhan Y, Duggal A. Different sorting algorithms comparison based upon the time complexity. *Int J Res Anal Rev*. 2020;3:114–21.
111. Gill SK, Singh VP, Sharma P, Kumar D. A comparative study of various sorting algorithms. *International Journal of Advanced Studies of Scientific Research*, 2019; 4(1).
112. Buradagunta S, Bodapati JD, Mundukur NB, Salma S. Performance comparison of sorting algorithms with random numbers as inputs. *Ingénierie Des Systèmes D Inf*. 2020;25(1):113–7.
113. Singh DP, Joshi I, Choudhary J. Survey of gpu based sorting algorithms. *Int J Parallel Prog*. 2018;46:1017–34.
114. Alhabboub Y, Almutairi F, Safhi M, Alqahtani Y, Almeedani A, Alguwaifi Y. Accelerating large-scale sorting through parallel algorithms. *J Comput Commun*. 2024;12(1):131–8.
115. Gupta SK, Singh DP, Choudhary J. New gpu sorting algorithm using sorted matrix. *Proc Comput Sci*. 2023;218:1682–91.
116. Kobus, R, Nelgen, J, Henkys, V, Schmidt, B. Faster Segmented Sort on GPUs. *European Conference on Parallel Processing*, Springer, 2023;664–678.
117. Jalilvand A, Banitaba FS, Estiri SN, Aygun S, Najafi MH. Sorting it out in hardware: A state-of-the-art survey. *ACM Transactions on Design Automation of Electronic Systems*, ACM New York, NY, 2023;

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.